

Compte rendu

Ouarezki Oussama Abde Rahim G3

Introduction

In this project, we aim to build a decision tree model to predict the survival of passengers on the Titanic based on various features such as age, gender, and class. Initially, a simple decision tree is created using default parameters. However, to improve the model's performance and avoid overfitting, we utilize **GridSearchCV** for hyperparameter tuning. This approach helps identify the best combination of parameters for the decision tree, resulting in a more accurate and efficient model. We will evaluate the performance of both the base and optimized models using metrics such as accuracy, precision, recall, and F1 score to assess their predictive power.

Question 1

Chargez le fichier titanic-passengers.csv, puis affichez un aperçu des données pour comprendre la structure et les valeurs manquantes éventuelles.

```
import pandas as pd

data = pd.read_csv("titanic-passengers.csv",sep=';')
data.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch
0	343	No	2	Collander, Mr. Erik Gustaf	male	28.0	0	0
1	76	No	3	Moen, Mr. Sigurd Hansen	male	25.0	0	0
2	641	No	3	Jensen, Mr. Hans Peder	male	20.0	0	0
3	568	No	3	Palsson, Mrs. Nils (Alma Cornelia Berglund)	female	29.0	0	4
4	672	No	1	Davidson, Mr. Thornton	male	31.0	1	0

As we can see a quick look of how the dataframe look like.

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PassengerId     891 non-null   int64
1   Survived        891 non-null   object
2   Pclass          891 non-null   int64
3   Name            891 non-null   object
4   Sex             891 non-null   object
5   Age            714 non-null   float64
6   SibSp           891 non-null   int64
7   Parch           891 non-null   int64
8   Ticket          891 non-null   object
9   Fare            891 non-null   float64
10  Cabin           204 non-null   object
11  Embarked        889 non-null   object
dtypes: float64(2), int64(4), object(6)
memory usage: 83.7+ KB
```

Question 2

Effectuez le prétraitement des données :

- Remplacez les valeurs manquantes dans la colonne Age par la moyenne de la colonne.
- Encodez la colonne Sex avec des valeurs numériques (1 pour male, 0 pour female).
- Remplacez les valeurs manquantes dans Cabin par la valeur "G6".
- Encodez la colonne Survived en valeurs numériques (1 pour Yes, 0 pour No).

This code fills missing values in the "Age" and "Cabin" columns, and converts the categorical values in the "Sex" and "Survived" columns into numeric values, preparing the dataset for training and testing the model.

```
data['Age'].fillna(data['Age'].mean(), inplace=True)
data['Sex'] = data['Sex'].map({'male': 1, 'female': 0})
data['Cabin'].fillna('G6', inplace=True)
data['Survived'] = data['Survived'].map({'Yes': 1, 'No': 0})
```

C:\Users\AL-Athir\AppData\Local\Temp\ipykernel_3284\3476700472.py:1: FutureWarning: A value is set for the inplace parameter to True, in place. The behavior will change in pandas 3.0. This inplace method will never work because the inter

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: va

```
data['Age'].fillna(data['Age'].mean(), inplace=True)
```

C:\Users\AL-Athir\AppData\Local\Temp\ipykernel_3284\3476700472.py:3: FutureWarning: A value :
The behavior will change in pandas 3.0. This inplace method will never work because the inter

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: va

```
data['Cabin'].fillna('G6', inplace=True)
```

Question 3

Sélectionnez les colonnes pertinentes pour X (caractéristiques) en excluant les colonnes non nécessaires comme Name, Cabin, Ticket, et Embarked. La colonne Survived sera utilisée comme cible (y).

```
X = data[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']]
y = data['Survived']
```

This code separates the data into **explanatory variables** (X), which include class, sex, age, siblings/spouses aboard, parents/children aboard, and fare, and the **target variable** (y), which is whether the passenger survived.

Question 4

Utilisez `train_test_split` de Scikit-learn pour diviser les données en ensemble d'entraînement et de test (20 % pour le test, 80 % pour l'entraînement).

This code splits the data into training and testing sets, where 80% of the data is used for training (`X_train` and `y_train`) and 20% is used for testing (`X_test` and `y_test`), with a fixed random seed for reproducibility.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Question 5

Construire un modèle d'arbre de décision :

- Importez la classe `DecisionTreeClassifier` depuis la bibliothèque `sklearn.tree`.
- Créez un modèle DT initial sans paramètres (paramètres par défaut), puis entraînez le modèle sur l'ensemble d'entraînement avec la méthode `.fit()`.

The following code imports the necessary functions from **Scikit-learn**, creates a **Decision-TreeClassifier** model, and fits it on the training data. This means that the model will learn from the features (**X_train**) and the target variable (**y_train**) in the training dataset to build a decision tree.

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()
model.fit(X_train, y_train)
```

`DecisionTreeClassifier()`

Question 6

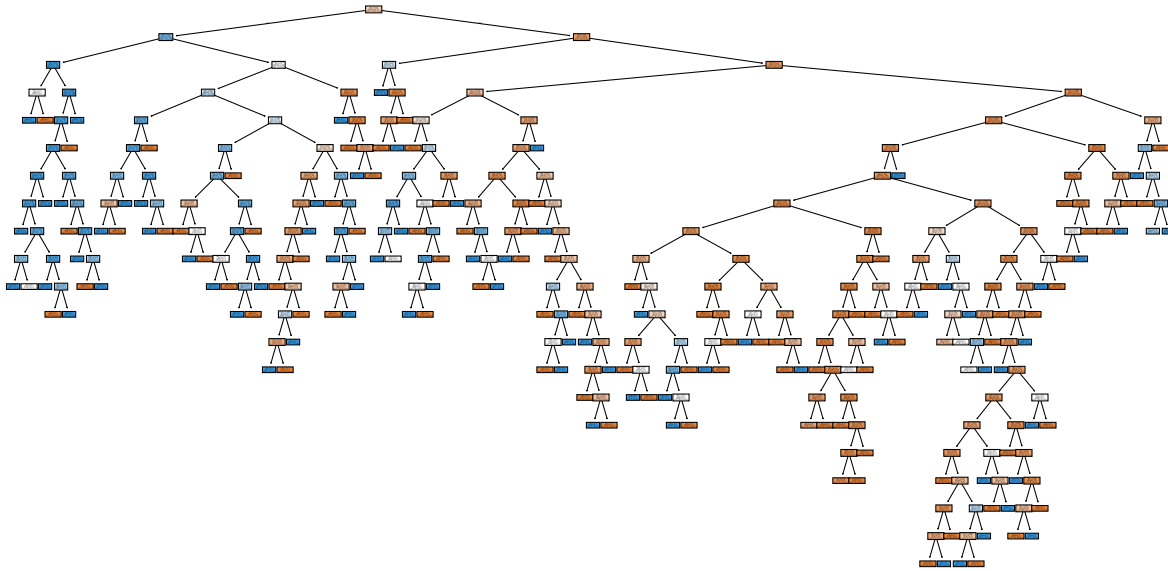
Utilisez `plot_tree` de Scikit-learn pour afficher l'arbre de décision. Ensuite, répondez aux questions suivantes :

- Expliquez brièvement le graphique obtenu.
- Mentionnez si l'arbre semble trop complexe (signe possible d'overfitting) ou trop simple (sous-ajustement).
- Analysez les règles apprises et leur pertinence pour le problème.

In this part, we're visualizing the decision tree that we just trained using **plot_tree** from **Scikit-learn**. This function will display the tree's structure, where each decision point corresponds to a specific feature of the passengers, like **Age**, **Fare**, or **Pclass**, and branches represent the conditions that split the data. By visualizing this, we can understand how the model makes predictions about survival based on these features.

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(20, 10))
plot_tree(model, filled=True, feature_names=X.columns, class_names=['Not Survived', 'Survived'])
plt.show()
```



Question 7

Utilisez le modèle que vous avez créé pour faire des prédictions sur l'ensemble de test et de train.

Ensuite, mesurez les métriques suivantes :

- Accuracy (Exactitude)
- Precision (Précision)
- Recall (Rappel)
- F1 Score (Score F1)

Sur les deux ensembles test et train, interprétez les résultats.

Now, I'll provide the Python code for making predictions and calculating these metrics:

We will use the trained model to predict the survival of passengers in both the **training** and **testing** datasets. After that, we will calculate the following metrics: **Accuracy**, **Precision**, **Recall**, and **F1 Score**. These metrics help evaluate the model's performance by comparing the predicted results with the actual results.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Make predictions
```

```

train_predictions = model.predict(X_train)
test_predictions = model.predict(X_test)

# Accuracy
train_accuracy = accuracy_score(y_train, train_predictions)
test_accuracy = accuracy_score(y_test, test_predictions)

# Precision
train_precision = precision_score(y_train, train_predictions)
test_precision = precision_score(y_test, test_predictions)

# Recall
train_recall = recall_score(y_train, train_predictions)
test_recall = recall_score(y_test, test_predictions)

# F1 Score
train_f1 = f1_score(y_train, train_predictions)
test_f1 = f1_score(y_test, test_predictions)

# Display the metrics
print(f"Train Accuracy: {train_accuracy}")
print(f"Test Accuracy: {test_accuracy}")
print(f"Train Precision: {train_precision}")
print(f"Test Precision: {test_precision}")
print(f"Train Recall: {train_recall}")
print(f"Test Recall: {test_recall}")
print(f"Train F1 Score: {train_f1}")
print(f"Test F1 Score: {test_f1}")

```

```

Train Accuracy: 0.9873595505617978
Test Accuracy: 0.7653631284916201
Train Precision: 0.9962962962962963
Test Precision: 0.6666666666666666
Train Recall: 0.9711191335740073
Test Recall: 0.7076923076923077
Train F1 Score: 0.9835466179159049
Test F1 Score: 0.6865671641791045

```

Explanation of the metrics

- **Accuracy (Exactitude)**: The proportion of correct predictions (both positive and negative) over all predictions made.

- **Precision (Précision)**: The proportion of true positive predictions relative to the total predicted positives. It tells us how many of the predicted survivors actually survived.
- **Recall (Rappel)**: The proportion of true positive predictions relative to the total actual positives. It tells us how many of the actual survivors were correctly predicted.
- **F1 Score (Score F1)**: The harmonic mean of Precision and Recall, balancing the two metrics when there's an uneven class distribution.

The interpretation of the model suggests overfitting, as the metrics on the training set are significantly better than those on the testing set. This disparity indicates that the model is likely performing well on the training data but struggling to generalize to unseen data. Overfitting can occur when the model is too complex relative to the amount of data available, causing it to memorize the training data instead of learning general patterns. This suggests that either the dataset is not large enough or the model's complexity exceeds the necessary level for accurate generalization.

In this case, the model's overfitting is due to the lack of parameter restrictions, which allowed it to become too complex and fit the training data excessively, limiting its generalization to new data.

Question 8

Construisez un second modèle d'arbre de décision en utilisant **GridSearchCV** pour optimiser les paramètres :

- Définissez une grille de paramètres à tester (par exemple, **max_depth**, **min_samples_split**, **min_samples_leaf**, **criterion**, etc.).
- Utilisez **GridSearchCV** pour rechercher les meilleures combinaisons de paramètres en fonction de la performance sur l'ensemble d'entraînement.
- Affichez les meilleurs paramètres trouvés.

We will use **GridSearchCV** from **Scikit-learn** to tune the hyperparameters of the decision tree model. GridSearchCV will evaluate all possible combinations of the parameters defined in the grid and select the best combination based on model performance (usually accuracy).

```
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier

my_parametres = {
    'criterion': ['gini', 'entropy'],
    'splitter': ['best', 'random'],
    'max_depth': [3, 4, 5],
    'min_samples_leaf': [1, 2, 5, 10],
```

```

    'max_leaf_nodes': [5, 10, 15],
}

my_model = DecisionTreeClassifier()

grid_search = GridSearchCV(estimator=my_model, param_grid=my_parametres, cv=10)
grid_search.fit(X_train, y_train)

print("Best parameters:", grid_search.best_params_)
print("Best score:", grid_search.best_score_)

```

Best parameters: {'criterion': 'entropy', 'max_depth': 4, 'max_leaf_nodes': 10, 'min_samples': 100}
 Best score: 0.8257824726134585

Explanation of the process:

1. **param_grid**: This is a dictionary where we define the parameters to tune. We include options like **max_depth**, **min_samples_split**, **min_samples_leaf**, and **criterion**.
2. **GridSearchCV**: This function performs an exhaustive search over the specified parameter grid. We use **cv=5**, meaning it will perform 5-fold cross-validation to evaluate each combination.
3. **Best parameters**: After fitting, we print the best combination of parameters found by GridSearchCV, which gives us the optimal configuration for the decision tree model.

Question 9

Reprenez les questions 5, 6, et 7 en appliquant cette fois le modèle optimisé via GridSearchCV.

```

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

best_model = grid_search.best_estimator_

y_train_pred = best_model.predict(X_train)
y_test_pred = best_model.predict(X_test)

train_accuracy = accuracy_score(y_train, y_train_pred)
train_precision = precision_score(y_train, y_train_pred)
train_recall = recall_score(y_train, y_train_pred)
train_f1 = f1_score(y_train, y_train_pred)

test_accuracy = accuracy_score(y_test, y_test_pred)

```



```

test_precision = precision_score(y_test, y_test_pred)
test_recall = recall_score(y_test, y_test_pred)
test_f1 = f1_score(y_test, y_test_pred)

print("Training set:")
print("Accuracy:", train_accuracy)
print("Precision:", train_precision)
print("Recall:", train_recall)
print("F1 Score:", train_f1)

print("\nTest set:")
print("Accuracy:", test_accuracy)
print("Precision:", test_precision)
print("Recall:", test_recall)
print("F1 Score:", test_f1)

```

Training set:
 Accuracy: 0.8328651685393258
 Precision: 0.8185483870967742
 Recall: 0.7328519855595668
 F1 Score: 0.7733333333333333

Test set:
 Accuracy: 0.8100558659217877
 Precision: 0.7540983606557377
 Recall: 0.7076923076923077
 F1 Score: 0.7301587301587301

Explanation of the code:

- **best_model:** We are using the optimized model found by **GridSearchCV** with **grid_search.best_estimator_**.
- **y_train_pred** and **y_test_pred:** The model makes predictions on the training and test sets, respectively.
- **accuracy_score**, **precision_score**, **recall_score**, **f1_score:** These functions measure the model's performance in terms of accuracy, precision, recall, and F1 score on both the training and test sets.

The results from these metrics will help interpret the performance of the optimized model on both datasets.

```

import matplotlib.pyplot as plt

# Data for plotting
metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
train_scores = [train_accuracy, train_precision, train_recall, train_f1]
test_scores = [test_accuracy, test_precision, test_recall, test_f1]

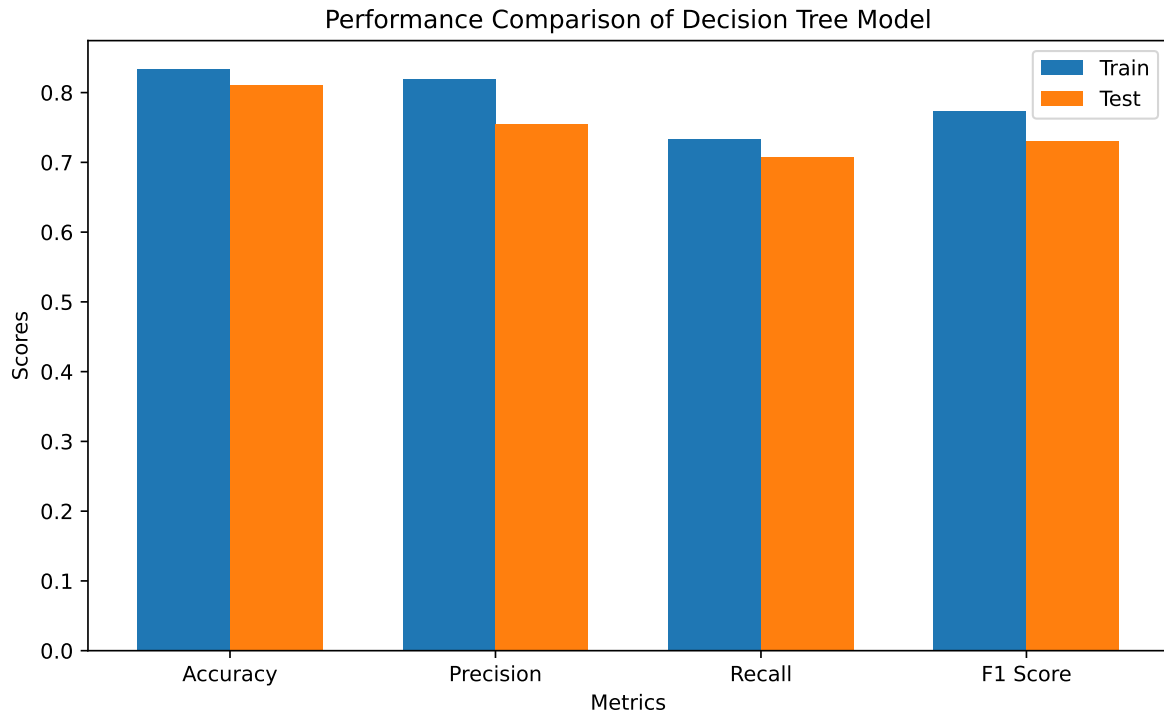
# Create a figure and axis for plotting
fig, ax = plt.subplots(figsize=(8, 5))

# Plot the data
bar_width = 0.35
index = range(len(metrics))
bar1 = ax.bar(index, train_scores, bar_width, label='Train')
bar2 = ax.bar([i + bar_width for i in index], test_scores, bar_width, label='Test')

# Adding labels and title
ax.set_xlabel('Metrics')
ax.set_ylabel('Scores')
ax.set_title('Performance Comparison of Decision Tree Model')
ax.set_xticks([i + bar_width / 2 for i in index])
ax.set_xticklabels(metrics)
ax.legend()

# Display the plot
plt.tight_layout()
plt.show()

```



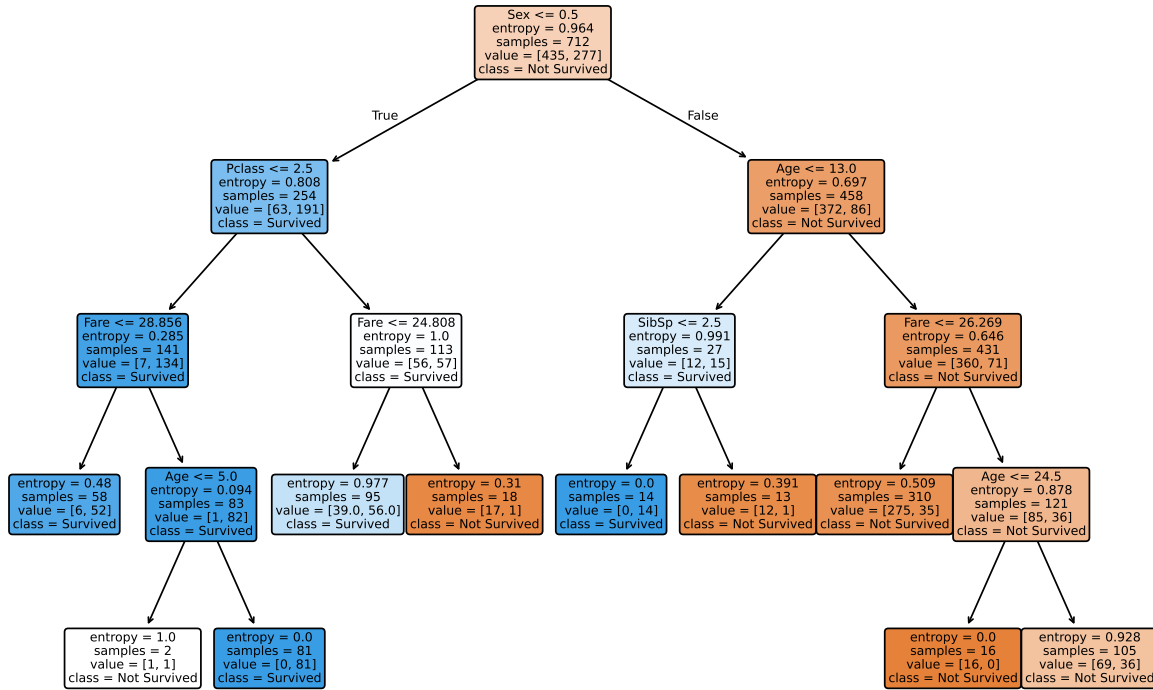
As we can see from the graph, the metrics on both the testing and training sets are almost the same. This indicates that there is no overfitting, as the model performs similarly on both datasets. The scores of the metrics are good, which suggests that the optimized model obtained through **GridSearchCV** performs better than the original decision tree model. The consistency between the training and test sets, along with the good performance metrics, shows that the model generalizes well to unseen data.

```
from sklearn.tree import plot_tree

# Fit the best model from GridSearchCV
best_model = grid_search.best_estimator_

# Plot the decision tree
plt.figure(figsize=(12, 8))
plot_tree(best_model, filled=True, feature_names=X_train.columns, class_names=['Not Survived', 'Survived'])
plt.title("Optimized Decision Tree")
plt.show()
```

Optimized Decision Tree



The new graph is less complex and more accurate than the previous one. This shows that the optimized model is not only simpler but also performs better, indicating that **GridSearchCV** helped improve the model's accuracy and generalization.

Conclusion

In conclusion, by using **GridSearchCV** to optimize the decision tree model, we achieved better performance compared to the initial model with default parameters. The optimized model demonstrated higher accuracy and consistency between the training and testing sets, suggesting it generalizes well without overfitting. The reduced complexity and improved metrics indicate that **GridSearchCV** effectively fine-tuned the decision tree, resulting in a more robust and reliable model for predicting survival outcomes.